

Technical Assignment – Task Tracker

Project Type: MERN Stack

Objective

Build a simple Task Tracker application with authentication and role based access. The system should support multiple user roles and allow users to create, assign, and track tasks through a basic workflow.\

Multi-Tenancy Support

The backend must support **Multi-Tenancy architecture**.

Requirements

- Each organization/company (tenant) should have isolated data access
 - Users, tasks, comments, and roles must belong to a specific tenant
 - A user from one tenant must never access another tenant's data
 - Use a scalable multi-tenant approach suitable for SaaS applications
 - Recommended approach:
 - Shared Database + Tenant ID isolation using middleware/filters/interceptors
 - Every request should carry tenant context:
 - JWT token
 - subdomain
 - or request header
 - Apply tenant filtering at repository/service level for all queries
 - Admin, Manager, and Member roles should work within their own tenant scope only
 - Structure the backend so it can later scale to:
 - schema-per-tenant
 - or database-per-tenant architecture later
-

Tech Stack

- **Frontend:** React (TypeScript preferred), React Router, state management of your choice
 - **Backend:** JS
 - **Database:** MongoDB
 - **API:** REST APIs (JSON)
 - **Bonus:** Docker for running DB + backend, Swagger/OpenAPI
-

Functional Requirements

Roles

Role	Permissions (High Level)
Admin	Can manage users, assign roles, view all tasks, update any task
Manager	Can create tasks, assign tasks to team members, view team tasks
Member	Can view tasks assigned to them, update task status, add comments

Authentication & Authorization

- Users must be able to sign up and log in
- Use JWT authentication
- All APIs must be protected based on role
- Store password securely using BCrypt

Task Management

- Create task with: title, description, priority, due date, status, createdBy, assignedTo
 - Statuses: TODO, IN_PROGRESS, DONE
 - Manager/Admin can assign tasks to Members
 - Member can only update status for tasks assigned to them
 - Admin can view and edit any task
-

Comments (Optional but Recommended)

Allow users to add comments on tasks. Comments should include:

- message
- createdBy
- createdAt

Members should be able to comment on tasks assigned to them; Managers/Admin can comment on any visible task.

UI Requirements (React)

- Login + Signup pages
- Dashboard based on role
- Task list page with filters:
 - status
 - priority
 - assignedTo
- Task detail view (with comments if implemented)
- Create/Edit task modal or page (Manager/Admin)
- User management page (Admin) – list users + change role

UI Notes

The UI does not need to be heavily styled, but should be clean and usable. Use any UI library (Material UI, Tailwind, Ant Design, etc.) or custom CSS.

Backend Requirements (JS/TS)

- Follow a clean structure:
 - controller → service → repository
 - Use DTOs for request/response objects
 - Validation for inputs (e.g., @NotBlank, @Email, etc.)
 - Global exception handling using @ControllerAdvice
 - Use Spring Security + JWT filters for auth
-

API Endpoints

Category	Endpoint	Method	Role Access
Auth	/api/auth/signup	POST	Public
Auth	/api/auth/login	POST	Public
Users	/api/users	GET	Admin
Users	/api/users/{id}/role	PUT	Admin
Tasks	/api/tasks	POST	Manager, Admin
Tasks	/api/tasks	GET	Member, Manager, Admin
Tasks	/api/tasks/{id}	GET	Member, Manager, Admin
Tasks	/api/tasks/{id}	PUT	Member (limited), Manager, Admin

Tasks	/api/tasks/{id}	DELETE	Manager, Admin
Comments	/api/tasks/{id}/comments	POST	Member, Manager, Admin
Comments	/api/tasks/{id}/comments	GET	Member, Manager, Admin

Database Design (Suggested)

You are free to design your own schema. A suggested structure is below:

users

- id
- name
- email
- passwordHash
- role
- createdAt

tasks

- id
- title
- description
- priority
- status
- dueDate
- createdById
- assignedToId
- createdAt
- updatedAt

comments

- id
 - taskId
 - message
 - createdById
 - createdAt
-

Submission Requirements

- Provide two separate repositories or a single monorepo with clear folders:
 - /frontend
 - /backend
 - Include a README.md with setup steps for frontend, backend, and database
 - Include sample credentials for Admin / Manager / Member (seeded users)
 - If using Docker, provide a docker-compose.yml for local setup
 - Push the code to GitHub and share the repository link
-

Evaluation Criteria

- Correctness and completeness of features
 - Role-based access control (frontend + backend)
 - Code quality, structure, and naming conventions
 - API design and error handling
 - UI usability and clarity
 - Database design and clean persistence logic
-

Bonus (Optional)

- Search tasks by title/description
 - Pagination for task list
 - Swagger/OpenAPI docs for backend
 - Unit tests (backend and/or frontend)
 - Deploy backend + frontend (Render/Vercel/Netlify) and share links
-